Deliverable 43 — Topic-derived scoring persona (builder-generated, agnostic) replaces the hardcoded “ESG/governance” analyst

Date: 2026-10-19
Author: CompanyIQ engineering
Commit: `8d2ced3`
Production SHA (verified live): `8d2ced3` — production **app** and **worker** both `SUCCESS` @ 2026-10-19T11:46:53Z
Verdict impact: NOT MEASURED — requires an authorized scoring batch (see §5). This is a deliberately verdict-affecting change; no “0 flips” is claimed.

---

1. What you asked for

> *Make the live scoring persona based on the topic for that framework rather than “ESG/governance”. So if the framework focuses on AI Management, the persona is an expert in that field. If this is applied in the framework builder stage, I think this is possible?*

Yes — and it is now done exactly that way. The persona is generated and stored by the **framework builder** when a framework is saved, and the analyzer consumes it at scoring time. The hardcoded `"expert ESG/governance analyst"` string is gone from the scoring path entirely. And yes — **creating a test framework to validate it sits with you** (the builder’s authoring flow runs real model calls on a real topic, which is your validation step; I can’t exercise that without spending on model calls, and you validate by creating one regardless).

---

2. How it works now (builder generates → analyzer consumes)

Builder stage (`server/routes/framework-builder-v2.ts`, `/v2/save`). When a framework is finalized, a `scoringPersona` is generated from the framework’s own topic and persisted on the framework record — right alongside where `topicTerm` / `topicSynonyms` are already derived. It uses the shared helper so the stored persona and the runtime fallback are identical in form. Nothing topic- or company-specific is hardcoded; the domain comes from the framework’s own topic.

Analyzer stage (`server/lib/analyzer.ts`). A new pure, exported helper `buildScoringPersona(framework)` replaces the hardcoded literal at **both** scoring prompts (the binary scorer ~L569 and the stricter partial scorer ~L725). Precedence:

1. If the framework has a stored `scoringPersona` → use it verbatim.
2. Else derive from the framework’s own topic: `topicTerm` ‖ `topicDescription` ‖ `name` → `"You are an expert <topic> analyst scoring corporate disclosures against a structured assessment framework."`

3. Else (no topic info at all) → neutral generic: `"You are an expert analyst scoring corporate disclosures against a structured assessment framework."`

The derived topic string is whitespace-collapsed and capped at 120 chars so it can't distort the prompt. **Everything else in both system prompts is byte-identical** — the `Topic:` line, base-rate block, SCORING RULES, CONFIDENCE LEVELS, anti-inference rules. Only the first persona sentence became dynamic.

So an **AI Management** framework now scores under *"You are an expert AI Management analyst..."*, a **Climate Transition** framework under *"...expert Climate Transition analyst..."*, and so on — with no per-framework code.

---

3. The default I set (tell me to change it if you disagree)

For **older frameworks that have no stored** `scoringPersona` (built before this change), the analyzer **derives** the persona from their own topic fields rather than falling back to the old "ESG/governance" string. This makes the whole app genuinely topic-agnostic immediately, rather than only for frameworks you rebuild. The consequence: an existing framework's scoring persona changes the next time it is scored (e.g. the current AI Management framework moves from the ESG persona to an AI-Management persona without needing a rebuild). That is the verdict-affecting part addressed in §5.

If you would prefer the safer "only newly-built frameworks change" behaviour, I can make the legacy fallback preserve the old ESG persona — say the word and it's a one-line change.

---

4. Files changed (5 files, +157 / −2)

File	Change
`shared/schema.ts`	New nullable `scoringPersona` text column on the frameworks table
`server/db.ts`	Idempotent `ALTER TABLE frameworks ADD COLUMN IF NOT EXISTS scoring_persona TEXT` (matches the existing `topic_term` / `topic_synonyms` pattern)
`server/lib/analyzer.ts`	New pure `buildScoringPersona(framework)`; both hardcoded personas replaced; ESG/governance literal removed
`server/routes/framework-builder-v2.ts`	`/v2/save` generates + persists `scoringPersona`
`server/lib/scoring-persona.test.ts`	New — 11 deterministic tests + a source guard against re-introducing the ESG literal

5. Verdict impact — honestly: NOT MEASURED

This change **alters the scoring system prompt** for existing frameworks whose persona was previously “ESG/governance”. On genuinely borderline measures, a reworded persona can flip yes↔no cells. Quantifying the flip count (the Yes→No / No→Yes totals this project reports) **requires running a scoring batch**, which spends money and was not authorized — so I did **not** run one, and I am **not** reporting a fabricated “0 flips”.

Metric	Value
Yes→No	not measured
No→Yes	not measured
Total flips	not measured

What this needs: an authorized scoring batch comparing the pre-change persona vs the topic-derived persona on a frozen corpus, to report the actual cell-flip totals. I can scope and run that on your go-ahead (it has a model-cost; I’ll confirm the estimate before running). Until then, the change is validated only by the deterministic gates below — which prove the logic is correct and agnostic, not that verdicts are unchanged.

---

6. Gates (all green at `8d2ced3`)

Gate	Result
`tsc --noEmit` (server)	62 pre-existing → **62** (0 new from changed files)
Builder + new persona tests	**381 / 381 pass** (370 prior + 11 new)
`grep -c "ESG/governance" server/lib/analyzer.ts`	**0** — literal fully removed
Client	No client code touched → no `vite build`
Deploy parity	Local HEAD == origin/main == production app == production worker == **`8d2ced3`**

Unit tests cover: stored persona returned verbatim; `topicTerm="AI Management"` → AI-Management persona; `topicDescription`-only derivation; no-topic → neutral generic; and an assertion that “ESG/governance” never appears in the AI/generic outputs (agnosticism regression guard).

---

7. Constraint compliance

1. **Code tested == production.** HEAD == origin/main == both production services == `8d2ced3`; tests ran against that tree.
2. **Yes→No / No→Yes totals.** Reported as **not measured** with the reason (§5) — a verdict-affecting change whose flip count needs an authorized batch; no false 0.
3. **Framework-level, not company/topic-specific.** Persona always derives from the framework's own topic fields; no hardcoded topic or company literal (regression-guarded by test).
4. **Relevant text never dropped.** Only the persona sentence changed; retrieval, evidence packing, trimming and all other prompt sections are untouched.
5. **Phase 2 — no human review of thousands.** Persona is generated automatically by the builder; no per-result human review added.

---

8. Your next step

Create a test framework in Framework Builder V2 on a non-ESG topic (e.g. AI Management). On save it will store a topic-matched `scoringPersona`; scoring that framework will run under the expert-in-that-field persona. If you want the verdict-flip totals for the existing frameworks quantified, tell me to scope the validation batch and I'll confirm the cost first.